

GuideChuna 프로그램 전체 운용 개념 및 흐름

문서 성격: 개발 인수인계·팀 내부용 런타임 흐름 설명서
범위: 앱 부팅부터 로비·인증 → 시나리오 실행 → 판정·채점 → 결과·서버 업로드 → 복귀까지 end-to-end
작성 기준일: 2026-07-22 (커밋 `4a2e06b` 기준 실제 코드 트레이싱)
관련 문서: `SingleScene_DataDriven_Architecture.md` (설계 의도), `Project_Structure_Analysis.md` (모듈별 사양), `EVALUATION_METRICS.md` (평가지표 정의)
⚠ 이 문서와 설계 문서의 차이: `SingleScene_DataDriven_Architecture.md` 는 02-26 설계안이라 일부 명칭이 실제 구현과 다릅니다(예: 설계=문자열 `scenarioId` + `Resources.Load` , 실제=정수 인덱스로 직렬화 배열 `scenarioConfigs[]` 선택). 본 문서는 현재 코드 실측 기준입니다.

1. 시스템 개관

GuideChuna는 Meta Quest용 VR 추나(현훈추나) 술기 훈련 키오스크 앱이다. 시나리오마다 별도 씬을 두지 않고 단일 통합 씬(`TrainingScene`) 에 `ScenarioConfig` 데이터를 주입해 환자 자세·애니메이션·시나리오 진행을 상황별로 구성하는 데이터 드리븐 구조다.

런타임은 3대 국면으로 나뉜다.

국면	씬	핵심 컨트롤러	역할
① 로비·인증·선택	<code>lobby.unity</code>	<code>LobbyAuthUI_Complete</code>	로그인, 시나리오 카드 선택 → <code>PlayerPrefs</code> 저장 후 씬 전환
② 시나리오 실행	<code>TrainingScene.unity</code>	<code>ScenarioBootstrapper</code> → <code>ScenarioManager</code> / <code>ScenarioConditionManager</code>	config 주입, CSV 로드, Phase→Step→SubStep 진행
③ 판정·결과·복귀	<code>TrainingScene.unity</code>	<code>ChunaPathEvaluator</code> / <code>EvaluationScoringEngine</code> / <code>TrainingResult*</code>	손포즈 유사도·한계 채점, 결과 표시·저장·서버 업로드, 로비 복귀

최상위 흐름도

[부팅]

| lobby.unity = 빌드 인덱스 0 → 자동 로드

▼

국면 ㉑ 로비

| LobbyAuthUI_Complete

| · 기기 인증 (AuthenticateDevice)

| · 조 선택 → 사용자 선택 → PerformLogin

| · 시나리오 카드 클릭

| └ 로그인 게이트 통과 시

| PlayerPrefs["SelectedScenario"] = idx

| SceneLoader.LoadScene("TrainingScene")

| (LoadingScene 경유 비동기 로드)

▼

국면 ㉒ 시나리오 실행

| ScenarioBootstrapper (실행순서 -100, Awake)

| · PlayerPrefs 인덱스 → scenarioConfigs[idx]

| · ScenarioManager.SetScenarioConfig(config)

| · 환자 프리셋/나레이션 폴더/제목 주입

| | [시작] 버튼

| ▼

| ScenarioManager.StartScenario()

| · CSV 로드(=scenarioName.csv) → Phase/Step/Sub

| · Animator 교체 · Cranial 리그 선택

| · 이벤트 체인 → SubStepStarted 발화

| |

| ▼ (substep마다 반복)

| SubStepStarted └ ScenarioManager (기계 셋업)

| └ ScenarioConditionManager

| (진행 엔진: conditionType

| 디스패치 · 폴링 · 완료)

| | 각 substep 완료 → NextSubStep

| ▼

| 마지막 substep → NextStep → NextPhase

| → CompleteScenario

▼

국면 ㉓ 판정·결과

| (실행 중 상시) ChunaPathEvaluator

| · HandPoseComparator 유사도 샘플링

| · ChunaLimitChecker 한계 상태

· EvaluationScoringEngine 스냅샷 누적
(종료 시) FinishTracking → FinalizeResult
· TrainingResultTextDisplay (VR 텍스트)
· TrainingResultExporter (로컬 CSV)
· TrainingResultUploader (POST /result/chuna)
ExitPopup [메인으로]
▼
SceneLoader.LoadScene(lobby) → 국면 ①로 복귀

2. 씬 구성과 부팅

- 키오스크는 코드 모드가 아니라 운용 개념이다. 코드베이스에 kiosk/autoStart/lockTask류 로직은 없다. Android 매니페스트도 표준 VR 런처(`launchMode="singleTask"` , `LAUNCHER` + `com.oculus.intent.category.VR`)일 뿐이다. → 실제 키오스크 잠금은 기기/런처 설정 영역(별도 문서 [키오스크_자동실행_문제해결_정리.md](#)).
- 부팅 씬 = `lobby.unity` (`EditorBuildSettings` 빌드 인덱스 0). 앱 실행 시 이 씬이 자동 로드된다.
- 씬 전환기 = `SceneLoader.LoadScene(sceneName, useLoadingScene=true)` (`UI/SceneLoader.cs`). `LoadingScene` 을 먼저 띄우고, 대상 씬을 `allowSceneActivation=false` 로 비동기 로드하다 진행률 ≥ 0.9 에서 `Resources.UnloadUnusedAssets()` + `GC.Collect()` (Quest 메모리 최적화) 후 활성화한다. `Camera X` 오브젝트는 `DontDestroyOnLoad` 로 씬 간 보존.
- 주요 씬: `lobby.unity` (로비, idx 0), `TrainingScene.unity` (통합 훈련 씬, idx 2), `LoadingScene` (로딩).

3. 국면 ① — 로비 · 인증 · 시나리오 선택

3.1 브레인: `LobbyAuthUI_Complete`

로비의 모든 것을 관장하는 중앙 컨트롤러(`Auth/LobbyAuthUI_Complete.cs`).

- 부팅 초기화: `Awake()` 에서 인증 서비스 헬퍼 UI 배선. `Start()` 에서 정적 `isFirstLaunch` 플래그로 분기 — 최초 실행이면 로그인 정보를 지워 **Guest** 상태로 시작(`ClearLoginInfo`), 로비 재진입이면 저장된 로그인 복원. 이어서 `LoadSavedDeviceSN()` + `AuthenticateDevice()` 로 기기 인증을 수행한다.

3.2 로그인 흐름 (시나리오 실행 전 필수)

```
사용자 아이콘 탭 → OnUserIconClicked()
└─ 미로그인 시: 조(grade) 선택 패널
    → OnGradeSelected → 사용자 선택 패널
    → OnUserSelected → PerformLogin(userId, username)
        └─ authFlowManager.PerformLogin(deviceSN, username)
            └─ currentUsername 설정
            └─ loginStateStore.SaveLoginInfo(username, userId)
            └─ 시나리오 카드 활성화
```

로그인 정보는 `LoginStateStore` 가 `PlayerPrefs`(`LOGIN_USERNAME` , `LOGIN_USERID` , `DEVICE_SN` , `ORG_ID`)에 영속화한다. 이 값들은 이후 국면 ③의 서버 업로드 payload로 재사용된다.

3.3 시나리오 카드 클릭 → 실행 진입점

핵심 진입점은 `OnScenarioCardClicked(int scenarioIndex)` .

1. 로그인 게이트: `currentUsername` 이 비어 있으면 `ShowLoginRequiredPopup()` 후 `return` . → 로그인 없이는 어떤 시나리오도 시작 불가.
2. `PlayerPrefs.SetInt(PrefsKeys.SelectedScenario, scenarioIndex)` + `PlayerPrefs.Save()`
3. `SceneLoader.LoadScene("TrainingScene")`

카드 → 진입점 어댑터 2종 (둘 다 같은 `OnScenarioCardClicked` 로 수렴):

어댑터	타입	위치	비고
<code>ScenarioCardButton</code>	구형 Button	<code>Auth/ScenarioCardButton.cs</code>	Range 0~4 레거시
<code>ScenarioLaunchButton</code>	신형 Toggle	<code>UI/ScenarioLaunchButton.cs</code>	현행 로비 카드. <code>useAdapterCards</code> 플래그로 활성화

`scenarioIndex` 는 `ScenarioBootstrapper.scenarioConfigs[]` 직렬화 배열의 평탄 인덱스다(0 상부승모근 ... 11 경추ROM측정, 12 두개골PJ교정). 별도 `ScenarioLauncher.LaunchScenario(index)` 도 동일한 저장+로드를 독립 수행하지만, 현행 카드는 브레인을 경유한다.

3.4 씬 간 계약 = PlayerPrefs

로비가 훈련 씬에 넘기는 유일한 계약은 **PlayerPrefs** 키다. (씬 간 직접 참조 없음 → 느슨한 결합)

키	타입	설정 위치	소비 위치	비고
<code>SelectedScenario</code>	int	<code>OnScenarioCardClicked</code>	<code>ScenarioBootstrapper</code>	★핵심. 배열 인덱스
<code>LOGIN_USERNAME</code> / <code>LOGIN_USERID</code>	string/int	<code>PerformLogin</code>	결과 업로드	사용자 식별
<code>DEVICE_SN</code> / <code>ORG_ID</code>	string	기기 인증	결과 업로드	기관/기기 식별
<code>SelectedMode</code> / <code>SelectedDifficulty</code>	string	<code>InfoPanelController</code>	(소프트 캐시)	실제 동작은 <code>DifficultyManager</code> 싱글톤이 결정

4. 국면 ② — 훈련 씬 부트스트랩 & 시나리오 실행 엔진

4.1 `ScenarioBootstrapper` — config 선택·주입

`[DefaultExecutionOrder(-100)]` 으로 씬에서 가장 먼저 실행. 배선은 전부 `Awake()`.

```

selectedIndex = forceDefaultIndex ? defaultScenarioIndex
                    : PlayerPrefs.GetInt("SelectedScenario", default)

config = scenarioConfigs[selectedIndex] // 범위 가드 후
scenarioManager.SetScenarioConfig(config) // 핵심 주입
+ AnatomyMuscleController.ApplyScenario
+ ScenarioConditionManager.SetNarrationScenarioFolder(narrationSubFolder ?? scenarioName)
+ InfoPanelController.SetScenarioTitle
+ PatientPositionManager.ApplyPreset(config.patientPositionPreset) // 씬 로드 시점
Start() 코루틴에서 1프레임 뒤 → scenarioManager.ApplyAnimatorController()

```

- `SetScenarioConfig` 안에서 `csvFileName = config.scenarioName` — 시나리오명이 곧 CSV 파일명이다. `ApplyEvaluationThresholds(config)` 도 적용.
- `forceDefaultIndex` 가 켜져 있으면 로비를 무시하고 `defaultScenarioIndex` 를 강제 로드(에디터 테스트용, 실사용 전 꺼야 함).

4.2 `ScenarioManager.StartScenario` — 시작

`StartScenario()` 는 [시작] 버튼(`InfoPanelController`)이 트리거한다.

```

StartScenario() → LoadFromCSV()

loader.LoadScenarios(csvFileName)          // Resources/Scenarios/{csvFileName}.csv
scenario = collection.scenarios[0]
ApplyEvaluationPhaseFilter(scenario)        // 평가모드일 때 config.evaluationPhases 밖 phase 제거
StartScenario(scenario)                    // 실제 초기화
    • 인덱스 리셋, advancedFromSubStep = null(재진입 가드 초기화)
    • SwitchAnimatorController()           // 환자 Animator = config.animatorController
    • ApplyCranialRigForScenario()          // 시나리오명 매칭 CranialAdjustmentController만 활성화
    • 결과 추적 시작
    • 이벤트 체인: ScenarioStarted → PhaseChanged → StepChanged → SubStepStarted

```

마지막 `SubStepStarted(currentSubStep)` 가 진행 루프의 첫 발화점이다.

4.3 데이터 계층

```

ScenarioConfig (ScriptableObject, Resources/ScenarioConfigs/{name}.asset)
    • scenarioName          ← CSV 파일명 + 표시명
    • animatorController    ← 환자 애니메이터
    • patientPositionPreset (lying/seated/...)
    • narrationSubFolder    (비면 scenarioName 폴백)
    • evaluationPhases[]    ← 평가모드 phase 필터
    • 접촉 대상/임계값 오버라이드

ScenarioData (CSV 파싱 결과, Resources/Scenarios/{name}.csv)
    ↳ PhaseData[]   phase = 시작 / 평가 / 교정 / 재평가 / 종료
        ↳ StepData[] IsGuideStep() = (stepNo == 0)
            ↳ SubStepData[]
                • conditionType / conditionParams
                • handTrackingFileName (손포즈 CSV)
                • voiceInstruction (나레이션 클립명)
                • patientAnimationClip / movementType
                • contactTarget / pivotTarget
                • duration (나레이션 OFF 시 폴백 타이머)

```

4.4 진행 루프 — `SubStepStarted` 의 2-구독 구조

각 substep 진입 시 `SubStepStarted` 이벤트가 두 구독자에게 병렬 전달된다. 역할 분리가 핵심이다.

구독자	역할	하는 일
<code>ScenarioManager.OnSubStepStartedForHandPose</code>	기계(mechanics) 셋업	접촉 대상·각도 표시·결과 추적 준비, conditionType별 조건 객체 등록
<code>ScenarioConditionManager.OnSubStepStarted</code>	진행(progression) 엔진	조건 폴링/나레이션/타이머로 완료 판정 후 <code>NextSubStep</code> 호출

ScenarioManager 쪽 디스패치: `IsCranialConditionType` → `HandleCranial` (cranial 조건 등록),
`PassiveStretch` → `HandlePassiveStretch`, `handTrackingFileName` 있음 → `HandleHandPoseTracking` (손포즈
CSV 로드 + `CheckpointPoseCondition` 등록), `patientAnimationClip` 만 → `HandleAutoPlayAnimation`.

4.5 conditionType 디스패치 (진행 엔진 핵심)

`ScenarioConditionManager.ProcessConditionByType` 의 `switch(conditionType)` . 조건키 = "
`{phase}_{step}_{subStepNo}`" .

★나레이션 우선 규칙 (`HasNarration()` = voiceInstruction 있음):

- HandPose / cranial* → `HandleNarrationThenHandPose` — 나레이션 재생 후 폴링 + 20초 타이머 시작.
- 그 외 → `HandleNarrationThenDuration` — 나레이션 재생 후 CSV `duration` 을 기다리지 않고 나레이션 끝
나면 즉시 `NextSubStep` . (즉 나레이션 길이 = 진행 타이밍, duration은 나레이션 OFF일 때만 폴백)

비(非)나레이션 switch:

conditionType	처리	완료 조건
<code>HandPose</code>	등록된 조건 폴링(<code>StartConditionCheck</code>)	손포즈 시퀀스 완료
<code>PassiveStretch</code> / <code>PatientAnimation</code>	<code>WaitForAutoPlayThenProgress</code>	환자 애니 AutoPlay 종 료
<code>Narration</code>	<code>PlayNarrationAndProgress</code>	나레이션 종료
<code>Duration</code>	<code>AutoProgressWithoutAlert(duration)</code>	시간 경과
<code>Manual</code>	토글 버튼 활성화	사용자 [다음]
<code>cranialTouch/Grip/Pressure/DepthBreath</code>	<code>IScenarioCondition</code> 게이트 폴링	두개골 술기별 판정(접 촉/파지/압력/자세·호 흡)
(빈칸)	로더가 자동판정 → 대개 <code>Narration</code> / <code>Manual</code>	—

두개골 게이트 클래스(`HandleCranial` 에서 생성·등록): `DiagnosisTouchCondition` (touch),
`GripPointCondition` (grip), `PressureCondition` (pressure), `BreathingCondition` (depthBreath). 모두
`IScenarioCondition` 이라 동일한 폴링 → `NextSubStep` 레일을 탄다.

20초 무진행 폴백: `ConditionCheckRoutine` 이 0.5초 간격으로 `IsConditionMet()` 폴링, `progressTimeout` (기본
20s) 초과 시 수동 [다음] 토글을 활성화하되 폴링은 계속(그 사이 동작을 완성하면 정상 진행).

4.6 substep 완료 → 진행 체인

```
ConditionCheckRoutine: IsConditionMet() == true
    → OnConditionCompleted()
        • 완료 사운드, 유사도 읽기
        • StepFeedbackUI 약 3초 표시
        • 나레이션 종료 대기
        • SceneManager.NextSubStep()
            ↳ [재진입 가드] advancedFromSubStep 로 substep당 1회만 전진
            ↳ 다음 substep SubStepStarted (루프)
            ↳ 없으면 NextStep → NextPhase → CompleteScenario
```

★재진입 가드 `advancedFromSubStep` : 나레이션-then-duration 경로와 AutoPlay 완료 핸들러가 각각 `NextSubStep` 을 부를 수 있어 **substep을 건너뛰는 이중 전진 버그**가 있었다. 가드로 substep당 정확히 1회 전진을 보장한다(전 시나리오 공통 회귀 수정, 07-15).

5. 국면 ③ — 판정 · 채점 · 결과 · 업로드

5.1 연습 모드 vs 평가 모드

모드는 `PlayerPrefs`가 아니라 `DifficultyManager.currentLevel` enum이 권위값이다.

- `DifficultyLevel { Beginner, Intermediate, Advanced, Evaluation }` — 앞 3개 = 연습, `Evaluation` = 평가.
- 선택 UI: `InfoPanelController.OnModeToggleChanged` . `PlayerPrefs["SelectedMode"]` 는 소프트 캐시일 뿐 동작을 좌우하지 않는다(런타임은 `DifficultyManager.Instance` 를 읽음).
- 동작 플래그: `IsEvaluationMode` / `IsOfficialScore` = `currentLevel == Evaluation` . 프리셋별 `ShowGuideHands` , `TrackAttempts` , `UnlimitedTime` , `SimilarityThreshold` 등.

채점은 두 모드 모두 돈다. 모드가 바뀌는 것은:

항목	연습	평가
공식 점수/업로드 취급	비공식	공식(<code>isOfficialEvaluation</code>)
중도 이탈 저장	버림	<code>isCompleted=false</code> 로 저장·업로드
결과 요약 포맷	<code>BuildPracticeSummaryText</code>	<code>BuildSummaryText</code>
phase 필터(스텝 게이팅)	미적용	<code>evaluationPhases</code> 로 제한
시도 횟수 키	<code>..._연습_...</code>	<code>..._평가_...</code>

⚠ **용어 혼동 주의:** `EvaluationPhaseManager` (§5.3)는 **substep** 단위 동작 상태기계이지 연습/평가 모드 선택기가 아니다. "Evaluation"이라는 두 개념을 구분할 것.

5.2 손 포즈 판정

```
HandPoseDataLoader.LoadFromFile(csv) → List<PoseFrame>    // 관절별 로컬 포즈 + 루트
ChunaPathEvaluator.CalculateCurrentSimilarity
    • userHandFrameIndex(사용자 진행)에 해당하는 가이드 프레임 선택
    • HandPoseComparator 로 비교
      [간이 경로, 기본]
        오른손(주동수) = 위치50% + 손바닥회전30% + 손모양20%
        왼손(보조수)   = 손바닥 하방 + 주먹 아님 평균
      [상세 경로]
        ComparePose: 관절 가중 유사도(손목×3, 손끝×1, 기타×0.5)
                      + 손바닥 월드 위치/회전 가중 혼합
    • passed = 유사도 >= 임계(기본 0.5), 연속 3프레임(consecutiveFramesRequired)부터 실인정
```

유사도는 **채점에 먹이는 값**이고, substep 전진 자체는 동작 상태기계(§5.3)의 hold 타이머 완료가 구동한다.

5.3 동작 상태기계 `EvaluationPhaseManager`

substep 하나의 동작 진행을 관리하는 상태기계: `WaitingForStart → StartHold → Moving → MidHold → Completed`. 채점 스냅샷은 `Moving / MidHold` /(등척성) `StartHold` 구간에서만 기록된다.

5.4 채점 `EvaluationScoringEngine`

```
(매 프레임, metricsRecordInterval 스로틀) RecordMetricsSnapshot
    • L/R 유사도 + 손 위치 + 왼손 접촉 여부 스냅샷 누적
    • ChunaLimitChecker 상태 접기:
      진행비율 ratio >= 0.5 → Exceeded, >= 0.3 → Warning, 그 외 Safe
      (왼손은 항상 Safe로 강제, 오른손=주동수만 채점)
    • Safe→Exceeded 진입 시에만 위반 카운트++, peakExceededRatio 추적

(substep 종료) CompleteEvaluation → CalculateFinalScore
점수 = 유사도블록(60) + 한계블록(40), clamp 0~100
    • 유사도블록 = averageSimilarity * 60
      (단일손 스텝은 주동수45 + 보조수15가 이미 60 안에 점함)
    • 한계블록 = 등척성: 40 * holdQuality
      그 외: 40 - (위반×2 + warningTime×0.5 + dangerTime×1 + exceededTime×3)
등급 = S≥95, A≥90, A≥85, B≥80, B≥75, C≥70, C≥65, D≥60, 그 외 F
→ OnEvaluationCompleted(session)
    → TrainingResultTracker 가 StepResult 로 접음 (가이드 스텝은 제외)
```

5.5 결과 수집 · 표시 · 저장 · 업로드

CompleteScenario() → resultTracker.FinishTracking() → FinalizeResult()

- overallSimilarity / overallScore / overallGrade 산출
- OnTrainingCompleted(resultData) 발화 → 3구독자

구독자	산출물	위치
TrainingResultTextDisplay	VR 내 결과 텍스트 (공식/연습 포맷 분기)	화면
TrainingResultExporter	로컬 CSV	Application.persistentDataPath/{폴더}/{시각}_사용자_{시나리오}_...csv
TrainingResultUploader	서버 업로드	아래 API

추가로 `SceneManager.ShowResultPanel()` 이 결과 웹뷰(`resultWebUrl`)를 브라우저 컴포넌트에 띄운다.

서버 업로드 API:

- 엔드포인트: `POST {baseApiUrl}/result/chuna`
base = `https://qpqjpivcg1.execute-api.ap-northeast-2.amazonaws.com`
- 전송: `UnityWebRequest` POST + `UploadHandlerRaw` JSON (`AuthenticationService.PostResultAsync`)
- payload `ResultData` : `orgID` / `userId` / `username` (로그인 `PlayerPrefs`), 고정 `subject="VR현훈추나"`, `learnModule` =시나리오명, `learnLevel1` = (평가) / (평가·미완료) / (연습), `learnLevel2` =스텝별 요약 문자열(약 750자), `runtime` =초.
- 가드: `sendOnEvaluationOnly` 가 켜져 있고 비공식이면 스킵(기본 false → 연습 결과도 기본 업로드). `orgID` 없거나 `userId<=0` 이면 업로드 스킵하되 로컬 CSV는 저장.

5.6 시도 횟수 저장

`TrainingAttemptStore` (`PlayerPrefs`), 키 = `ATTEMPT_COUNT_{평가|연습}_{시나리오}`. `IncrementAndGet` 을 `StartTracking` 에서 호출해 `attemptNumber` 에 반영. 기기 스코프라 로그아웃해도 유지된다.

5.7 복귀 흐름

`ExitPopupController` 의 팝업(다시하기 / 메인으로):

- 메인으로 → `SceneLoader.LoadScene(mainMenuSceneName)` = 로비(국면 ①로 복귀).
- 다시하기 → 현재 씬 리로드.
- 중도 이탈 안전망: 이탈/재시작/앱종료 시 `SaveIncompleteResultIfTracking` 호출 — 평가만 미완료로 저장. 업로드, 연습은 버림.

6. 핵심 데이터 계약 요약

계약	형태	규약
씬 간 전달	PlayerPrefs <code>SelectedScenario</code> (int) 외	\$3.4 표
시나리오 선택	<code>scenarioConfigs[]</code> 배열 인덱스	로비 카드 <code>scenarioIndex</code> ↔ Bootstrapper 배열 순서 일치 필수
CSV 위치	<code>Resources/Scenarios/{scenarioName}.csv</code>	파일명 = config.scenarioName
config 위치	<code>Resources/ScenarioConfigs/{name}.asset</code>	Bootstrapper 배열에 guid로 등록
나레이션	<code>Resources/Narrations/{Beginner Intermediate}/{폴더}/{voiceInstruction}</code>	폴더=narrationSubFolder(비면 scenarioName), 없으면 루트 폴백
손포즈	<code>handTrackingFileName</code> → 손포즈 CSV	HandPose 스텝만
서버	<code>POST /result/chuna</code>	\$5.5

7. 신규 시나리오 추가 체크리스트 (인수인계 실무)

새 술기 하나를 붙일 때 손대는 자리(두개골PJ교정 추가 시 실측 경로).

- CSV** `Resources/Scenarios/{이름}.csv` — 20컬럼, phase=시작/평가/교정/재평가/종료. 게이트 없는 walkthrough면 `conditionType` 비움(로더가 Narration 자동판정).
- config** `Resources/ScenarioConfigs/{이름}.asset` (+ `.meta` 새 guid) — `scenarioName` =CSV 파일명, `animatorController`, `patientPositionPreset`.
- 나레이션** `Resources/Narrations/{Beginner,Intermediate}/{이름}/{voiceInstruction}.mp3` — CSV voiceInstruction과 1:1. 시작/종료는 루트 폴백 가능.
- 등록** `TrainingScene.unity` 의 `ScenarioBootstrapper.scenarioConfigs[]` 에 config guid 추가 → 그 순번이 인덱스.
- 로비 카드** `lobby.unity` 에 카드 추가 → `ScenarioLaunchButton.scenarioIndex` = 위 순번, 라벨 지정. (씬 텍스트 편집 손상 위험 → **Unity** 인스펙터에서 기존 카드 복제 권장)

판정이 필요한 술기라도 새 `conditionType`을 만들 필요는 거의 없다 — 방향/접촉/포즈 판정은 기존 `HandPose` · `PassiveStretch` · cranial 게이트로 커버된다. 채점이 나오는 경로는 사실상 `HandPose` 하나뿐(손포즈 CSV 녹화가 최대 병목).

8. 핵심 파일 색인

국면	파일	역할
①	Auth/LobbyAuthUI_Complete.cs	로비 브레인(인증·로그인·카드 클릭)
①	Auth/LoginStateStore.cs	로그인 PlayerPrefs 영속화
①	UI/ScenarioLaunchButton.cs / Auth/ScenarioCardButton.cs	카드 어댑터
①↔②	UI/SceneLoader.cs	로딩씬 경유 비동기 씬 전환
②	Scenario/ScenarioBootstrapper.cs	config 선택·주입(실행순서 -100)
②	Scenario/ScenarioConfig.cs	시나리오 설정 ScriptableObject
②	Scenario/ScenarioManager.cs	Phase→Step→SubStep 진행, 기계 셋업
②	Scenario/ScenarioConditionManager.cs	conditionType 진행 엔진
②	Scenario/ScenarioCSVLoader.cs / ScenarioDataClasses.cs	CSV 파싱·데이터 계층
③	ChunaData/ChunaPathEvaluator.cs	손포즈 평가·스냅샷 오케스트레이션
③	PoseData/HandPoseComparator.cs / HandPoseDataLoader.cs	유사도 비교·가이드 로드
③	ChunaData/Helpers/EvaluationScoringEngine.cs	최종 점수(60/40)·등급
③	ChunaData/Helpers/EvaluationPhaseManager.cs	substep 동작 상태기계
③	ChunaData/ChunaLimitChecker.cs	한계 상태(Safe/Warning/Exceeded)
③	Training/DifficultyManager.cs	연습/평가 모드·난이도
③	Result/TrainingResultTracker.cs 외 TrainingResult*	결과 수집·표시·저장·업로드
③	Result/TrainingAttemptStore.cs	시도 횟수
③	Auth/AuthenticationService_Final.cs	서버 API(/result/chuna)
③	UI/ExitPopupController.cs	결과 후 복귀/재시작

본 문서는 커밋 `4a2e06b` 시점 코드 트레이싱 결과다. 파일·라인 상세는 각 클래스 참조.